

CodeCareer

En CodeCareer, desarrollé una plataforma escalable y eficiente para conectar profesionales tecnológicos con oportunidades laborales. Utilicé Node.js y MongoDB, lo que me permitió manejar grandes volúmenes de datos no estructurados y facilitar la escalabilidad horizontal. Apliqué validaciones en el servidor para mejorar el rendimiento y aseguré una experiencia de usuario fluida con funcionalidades como la subida de archivos con previsualización y la gestión eficiente de consultas.

1. Optimización de rendimiento y escalabilidad:

Una de las decisiones clave fue utilizar **MongoDB** como base de datos. Esto me permitió manejar grandes volúmenes de datos no estructurados, que es ideal para el tipo de aplicación que estoy desarrollando, donde las clases y vacantes pueden variar mucho en contenido. MongoDB facilita la escalabilidad horizontal, lo que significa que si la plataforma crece, puedo añadir más servidores para distribuir la carga sin comprometer el rendimiento.

En términos de validación, utilicé **express-validator** en el servidor para asegurar que los datos que se envían sean correctos y completos. Esto mejora el rendimiento porque evita que el servidor procese solicitudes que podrían estar mal formadas o incompletas, lo que también optimiza la experiencia del usuario, ya que los errores se gestionan antes de llegar al servidor.

En cuanto a la gestión de archivos, implementé un sistema de subida de archivos con una barra de progreso y previsualización en el frontend. Aunque no hay una optimización avanzada en términos de compresión de imágenes o minificación de CSS, esto mejora la experiencia del usuario al permitir que verifiquen los archivos antes de enviarlos.

Si bien no implementé **caching** ni paginación en las consultas a la base de datos, **MongoDB** y **Mongoose** ya proporcionan un manejo eficiente de los datos, y la estructura modular de la aplicación permite mejorar el rendimiento de manera controlada conforme la plataforma crece.

Este enfoque me ha permitido optimizar la aplicación de manera que sea escalable y eficiente, asegurando que el rendimiento no se vea comprometido a medida que la plataforma crece.

2. Arquitectura modular

En cuanto a la **arquitectura modular** de CodeCareer, uno de mis principales enfoques fue asegurarme de que cada parte del sistema estuviera bien separada y organizada. Esto no solo mejora la claridad del código, sino que también facilita el mantenimiento y la escalabilidad del proyecto a medida que crece.

Primero, estructuré la aplicación dividiendo las responsabilidades en **controladores, modelos y rutas**:

1. Controladores: Aquí es donde coloqué toda la lógica de negocio. Por ejemplo, el controlador de **vacantes** gestiona todo lo relacionado con la **creación, edición y eliminación** de vacantes, mientras que el controlador de **usuarios** maneja el **registro, inicio de sesión y edición** de perfiles. Esta separación asegura que la lógica de cada módulo esté encapsulada y que no se mezcle con otros componentes.

2. Modelos: Utilicé **Mongoose** para definir los esquemas de los datos. Los modelos están diseñados para interactuar con la base de datos MongoDB. Cada entidad principal del sistema, como usuarios y vacantes, tiene su propio modelo, lo que permite trabajar con los datos de forma estructurada y modular.

3. Rutas: Configuré las rutas de manera que cada grupo de funcionalidad tenga su propio conjunto de rutas. Por ejemplo, las rutas relacionadas con las vacantes están en un archivo separado del que maneja la autenticación de usuarios. Esto hace que sea fácil añadir o modificar funcionalidades sin alterar otras partes del sistema.

Al mantener una **arquitectura modular**, puedo escalar fácilmente la aplicación. Si en algún momento se decide añadir una nueva funcionalidad, como un módulo de notificaciones, simplemente podría crear un nuevo controlador, modelo y conjunto de rutas específicos para esta funcionalidad, sin afectar el resto del sistema.

Además, esta modularidad también facilita la colaboración en equipo, ya que otros desarrolladores pueden trabajar en diferentes módulos de la aplicación sin interferir en otras partes. Cada módulo se comunica a través de interfaces bien definidas, lo que reduce el riesgo de introducir errores al añadir nuevas características o corregir bugs.

Este enfoque no solo mejora el mantenimiento y la claridad del código, sino que prepara la aplicación para crecer de manera controlada y organizada, sin volverse difícil de gestionar.

3. Ciclo completo de desarrollo

Cuando hablo del ciclo completo de desarrollo en CodeCareer, me refiero a que estuve involucrado en cada una de las etapas del proyecto, desde la planificación inicial hasta el despliegue en producción y el mantenimiento continuo. Este enfoque me permitió tener una visión integral del producto y asegurarme de que todas las fases estuvieran alineadas con los objetivos del proyecto.

3.1. Planificación y definición de requisitos

Comencé definiendo las funcionalidades clave de la plataforma, como la **gestión** de vacantes y la **autenticación** de usuarios. Esta fase también incluyó planificar la arquitectura para que fuera modular y escalable desde el inicio.

3.2. Diseño y arquitectura

Diseñé una arquitectura clara, usando MongoDB para la base de datos, Express.js como framework y organizando el código en módulos (controladores, modelos y rutas). El objetivo fue asegurar flexibilidad y escalabilidad a futuro.

3.3 Desarrollo

Desarrollé cada parte de la aplicación de manera iterativa, comenzando por las funcionalidades centrales como la creación de vacantes y la autenticación. Implementé validaciones y sanitización en los controladores y modelos con Mongoose.

3.4. Despliegue

Desplegué la aplicación en Heroku, configurando variables de entorno y optimizando para que el sistema pudiera escalar fácilmente con más usuarios. Esto garantiza que la aplicación funcione eficientemente en producción.

4. Trabajo con grandes volúmenes de datos

En CodeCareer, el manejo de grandes volúmenes de datos es un aspecto crucial, especialmente cuando se trata de vacantes y candidatos. Para garantizar que el sistema funcione de manera eficiente, implementé varias estrategias.

1. Uso de MongoDB

Elegí MongoDB como base de datos porque es altamente eficiente para manejar grandes volúmenes de datos no estructurados, como las vacantes y perfiles de los usuarios. MongoDB permite almacenar documentos de manera flexible, lo cual es perfecto cuando tienes múltiples vacantes con diferentes atributos.

2. Consultas optimizadas

Para optimizar el rendimiento de las consultas, diseñé los esquemas de **Mongoose** de manera que los campos más utilizados, como el título de las vacantes y las categorías, estuvieran indexados. Esto garantiza que las búsquedas sean rápidas, incluso cuando la base de datos crece en tamaño. Utilizar índices también evita que las consultas recorran toda la colección, reduciendo significativamente el tiempo de respuesta.

3. Almacenamiento de sesiones

Para gestionar a los usuarios concurrentes, almacené las sesiones en **MongoStore**, lo que me permite manejar eficientemente un gran número de usuarios que acceden al sistema al mismo tiempo. MongoStore, al estar vinculado a MongoDB, puede escalar junto con el crecimiento de los usuarios sin comprometer el rendimiento.

5. Validación de datos

En CodeCareer, la validación de datos es un aspecto fundamental para garantizar que la información ingresada por los usuarios sea correcta y segura. Implementé varias estrategias de validación tanto en el servidor como en el cliente.

5.1. Validación en el servidor con Mongoose

Utilicé Mongoose para manejar la validación de los esquemas en el servidor. Cada modelo tiene reglas de validación específicas que aseguran que los datos ingresados cumplen con ciertos requisitos.

Por ejemplo:

- En el modelo de Usuarios, me aseguré de validar que los correos electrónicos fueran únicos y estuvieran en un formato correcto.
- En el modelo de Vacantes, los campos como el título de la vacante, la descripción y el salario se validan para asegurarse de que no estén vacíos y cumplan con los formatos establecidos.

5.2. Validación en los controladores

Además de la validación proporcionada por Mongoose, añadí una capa adicional de validación en los controladores. Aquí, me aseguré de validar y sanitizar los datos antes de procesarlos, previniendo ataques como injection attacks o la introducción de datos maliciosos.

Utilicé herramientas como **express-validator** para esta tarea.

Por ejemplo, al crear una vacante, se valida que los campos obligatorios estén completos y que los datos ingresados en campos como salario o ubicación tengan el formato correcto antes de guardarlos en la base de datos.

5.3. Validación en el lado del cliente

En el lado del cliente, implementé validación en los formularios utilizando JavaScript. Los formularios de registro, inicio de sesión y creación de vacantes incluyen validaciones básicas, como la obligatoriedad de los campos, el formato correcto de los correos electrónicos y la coincidencia de contraseñas. Esto mejora la experiencia del usuario al proporcionar feedback inmediato cuando los datos no cumplen con los requisitos antes de enviarse al servidor.

5.4. Manejo de errores y mensajes de validación

Cuando los usuarios ingresan datos incorrectos, el sistema muestra mensajes claros e informativos, tanto en el lado del cliente como en el servidor. Utilicé mensajes flash para notificar a los usuarios sobre los errores en sus formularios, asegurándome de que tengan la oportunidad de corregir cualquier problema antes de reintentar.

Con este enfoque de validación en capas, garantizo que los datos ingresados sean seguros, correctos y cumplan con los requisitos de cada funcionalidad. Esto no solo mejora la experiencia del usuario, sino que también protege la integridad de la plataforma.